

TaskFlow

Real-Time Project Management SaaS

Software Requirements Specification

Version 1.0
April 2026
Portfolio Project — Full-Stack Web Development

Document Property	Detail
Project Name	TaskFlow — Real-Time Project Management SaaS
Document Type	Software Requirements Specification (SRS)
Version	1.0
Status	Draft — Ready for Development
Author	Biswash Khanal(Portfolio Project)
Target Stack	React + Vite, TypeScript, Express, PostgreSQL, Redis, Docker
Estimated Duration	6–10 weeks (solo developer)

1. Introduction

1.1 Purpose

This document defines the complete requirements for TaskFlow, a real-time project management web application built as a portfolio project. It covers functional requirements, system architecture, data models, API contracts, and implementation guidelines. The intended audience is the developer (sole author) and any technical reviewers evaluating the project.

1.2 Project Overview

TaskFlow is a multi-workspace task tracking application with real-time collaboration features. It allows teams to organise work into projects, manage tasks on a kanban board, and see live updates as teammates make changes — without a page refresh. The scope is intentionally similar to tools like Linear or Jira, but scoped to what a single developer can deliver to a high standard within 6–10 weeks.

The project is designed to demonstrate production-grade full-stack skills including: state management architecture, WebSocket-based real-time updates, role-based access control, containerisation with Docker, CI with GitHub Actions, and a clean REST API with proper authentication.

1.3 Scope

In scope for this version:

- User registration, login, and JWT-based session management
- Workspace and project creation and management
- Kanban board with drag-and-drop task management
- Real-time collaboration via WebSockets (Socket.io)
- Role-based access control (Owner, Member, Viewer)
- Task comments, labels, assignees, due dates, and priority
- Activity/audit log per task
- In-app notifications when assigned or mentioned
- Full-text task search and multi-filter support
- Dockerised development and production environment

Out of scope (explicitly not built):

- Email sending / SMTP integration
- File attachments and media uploads
- Billing, subscriptions, or payment integration
- Mobile native app
- OAuth / social login
- Gantt chart / timeline view

1.4 Definitions

Term	Definition
Workspace	The top-level container. A user can belong to multiple workspaces. Equivalent to an organisation.
Project	A grouping of related tasks within a workspace. Has its own kanban board.
Task	The core work item. Has status, priority, assignee, due date, labels, and comments.
Member	A user who belongs to a workspace with a specific role.
Role	Permission level: Owner, Member, or Viewer. Scoped to a workspace.
RBAC	Role-Based Access Control — restricting actions based on user role.
JWT	JSON Web Token — used for stateless authentication.
Socket.io	Library for WebSocket-based real-time bidirectional communication.
Zustand	Lightweight React state management library for client UI state.
React Query	Server state management library handling caching, fetching, and syncing.
Prisma	TypeScript ORM used to interact with PostgreSQL.
Redis	In-memory data store used for session cache and notification queuing.

2. Technology Stack

2.1 Stack Overview

Layer	Technology	Purpose	New to Developer?
Frontend	React 18 + Vite + TypeScript	UI framework and build tool	No
Frontend	Zustand	Client-side UI state management	Yes — learn this
Frontend	React Query (TanStack)	Server state, caching, sync	Yes — learn this
Frontend	Tailwind CSS	Utility-first styling	No
Frontend	dnd-kit	Accessible drag-and-drop for kanban	Yes — learn this
Backend	Node.js + Express + TypeScript	REST API server	No
Backend	Socket.io	WebSocket real-time layer	Yes — learn this
Backend	Prisma ORM	Database access layer	Yes — learn this
Auth	JWT (access + refresh tokens)	Stateless authentication	Partially new
Database	PostgreSQL 15	Primary relational database	No
Cache	Redis 7	Session cache + notification queue	Yes — learn this
DevOps	Docker + Docker Compose	Containerisation	Yes — learn this
DevOps	GitHub Actions	CI pipeline	Partially new

Note: For each 'Yes — learn this' item, read the official documentation first before using AI to fill gaps. Zustand, React Query, Socket.io, and Prisma all have excellent docs and short learning curves for someone with your background.

2.2 State Management Architecture

A common mistake junior developers make is putting everything into one state solution. This project uses two libraries with a clear division of responsibility:

Zustand — UI State Only

Zustand manages state that lives on the client and has nothing to do with the server. Examples:

- Whether the sidebar is open or collapsed
- Which task detail modal is currently open
- Active filters selected on the board (before they trigger an API call)
- Notification panel open/closed
- Current workspace/project selected in the UI

React Query — Server State Only

React Query manages everything that comes from or goes to the API. It handles caching, background refetching, loading/error states, and optimistic updates. Examples:

- List of tasks for a project
- Current user profile
- Workspace members list
- Notifications list

The rule is simple: if the state needs to be fetched from the server or saved to the server, it belongs in React Query. If it is purely a UI concern (open/closed, selected, hovered), it belongs in Zustand.

3. System Architecture

3.1 High-Level Architecture

TaskFlow follows a standard three-tier architecture with a separate real-time layer:

Tier	Component	Description
Client	React SPA (Vite)	Single page app served as static files. Communicates with API over HTTPS and with the real-time server over WebSocket.
Server	Express REST API	Handles all business logic, authentication, and database operations. Stateless — scales horizontally.
Server	Socket.io Server	Runs alongside Express on the same Node process. Manages WebSocket rooms per workspace.
Data	PostgreSQL	Primary persistent store. All application data: users, workspaces, tasks, comments, activity logs.
Data	Redis	Stores refresh token session data and acts as a message bus for broadcasting Socket.io events across future server instances.

3.2 Repository Structure

The project is a monorepo with two packages. This is not a full Turborepo/Nx setup — just a simple top-level separation:

```
taskflow/├─ apps/│   └─ frontend/      React + Vite app│   │   └─ src/│   │   │   └─ components/│   │   │   │   └─ features/  Feature-sliced modules (board, auth, etc.)│   │   │   │   │   └─ hooks/   Custom React hooks│   │   │   │   │   │   └─ lib/    API client, socket client, utils│   │   │   │   │   │   │   └─ store/  Zustand stores│   │   │   │   │   │   │   │   └─ types/│   │   │   │   │   │   │   │   └─ Shared TypeScript types│   │   │   │   │   │   │   │   │   └─ vite.config.ts│   │   │   │   │   │   │   │   │   └─ backend/  Express│   │   │   │   │   │   │   │   │   └─ API│   │   │   │   │   │   │   │   │   └─ src/│   │   │   │   │   │   │   │   │   └─ controllers/│   │   │   │   │   │   │   │   │   └─ routes/  Express route handlers│   │   │   │   │   │   │   │   │   └─ middleware/ Auth, validation, error handling│   │   │   │   │   │   │   │   │   └─ services/  Business logic layer│   │   │   │   │   │   │   │   │   └─ handlers│   │   │   │   │   │   │   │   │   └─ prisma/  Schema and migrations│   │   │   │   │   │   │   │   │   └─ socket/  Socket.io event│   │   │   │   │   │   │   │   │   └─ docker-compose.yml│   │   │   │   │   │   │   │   │   └─ docker-compose.prod.yml│   │   │   │   │   │   │   │   │   └─ .github/workflows/ci.yml│   │   │   │   │   │   │   │   │   └─ tsconfig.json
```

3.3 Docker Compose Setup

docker-compose.yml (development) defines four services:

- postgres — PostgreSQL 15 container with volume for data persistence
- redis — Redis 7 Alpine container
- backend — Node.js app with volume mounts for hot reload (ts-node-dev)
- frontend — Vite dev server with HMR enabled

docker-compose.prod.yml overrides for production: builds optimised images, removes dev volumes, uses environment secrets from .env.

Note: *Start with Docker Compose early — it removes the 'works on my machine' problem from day one. Every collaborator (or interviewer trying to run your project) just runs docker compose up.*

4. Functional Requirements

4.1 Authentication

ID	Requirement	Priority
AUTH-01	User can register with name, email, and password. Password hashed with bcrypt (cost factor 12).	Must have
AUTH-02	User can log in with email and password. Returns access token (15 min TTL) and refresh token (7 day TTL).	Must have
AUTH-03	Access token sent in Authorization header (Bearer). Refresh token stored as HTTP-only cookie.	Must have
AUTH-04	Dedicated /auth/refresh endpoint issues a new access token when called with a valid refresh token cookie.	Must have
AUTH-05	Logout endpoint invalidates the refresh token in Redis.	Must have
AUTH-06	All API routes except /auth/* require a valid access token. Return 401 on missing/expired token.	Must have
AUTH-07	Rate limit login endpoint: max 10 attempts per IP per 15 minutes. Return 429 on excess.	Should have

4.2 Workspace Management

ID	Requirement	Priority
WS-01	Authenticated user can create a new workspace. Creator is assigned Owner role automatically.	Must have
WS-02	Owner can invite users to the workspace by email. Invited user receives an invite token (or manual accept for MVP).	Must have
WS-03	Owner can assign roles: Owner, Member, Viewer. A workspace must always have at least one Owner.	Must have
WS-04	Owner can remove members from the workspace.	Must have
WS-05	Owner can delete the workspace. Cascades: deletes all projects and tasks.	Must have
WS-06	User can belong to multiple workspaces. UI shows a workspace switcher.	Must have
WS-07	Owner can update workspace name and avatar (text initials for MVP, no image upload).	Should have

4.3 Project Management

ID	Requirement	Priority
PROJ-01	Owner and Member can create a project within a workspace. Viewer cannot.	Must have
PROJ-02	Each project has a name, description, and optional colour identifier.	Must have
PROJ-03	Owner can delete a project. Member can only delete projects they created.	Must have
PROJ-04	Project has configurable status columns (default: Backlog, Todo, In Progress, In Review, Done).	Must have
PROJ-05	Owner can add, rename, reorder, or delete status columns.	Should have

4.4 Task Management

ID	Requirement	Priority
TASK-01	Owner and Member can create a task within a project. Viewer cannot.	Must have
TASK-02	Task fields: title (required), description (rich text / markdown), status, priority (None/Low/Medium/High/Urgent), assignee (single user), due date, labels (multi-select).	Must have
TASK-03	Task can be moved between status columns via drag-and-drop on the kanban board.	Must have
TASK-04	Task order within a column is persisted. Uses a fractional indexing strategy (e.g. lexicorank) to avoid full reorder on every drag.	Must have
TASK-05	Owner or task creator can delete a task.	Must have
TASK-06	Users can add comments to a task. Comments support markdown.	Must have
TASK-07	Every task change (field edit, status change, comment, assignment) creates an activity log entry.	Must have
TASK-08	Workspace members can be @mentioned in comments. Triggers a notification.	Should have
TASK-09	Tasks can be filtered by: assignee, label, priority, due date range. Filters are combinable.	Must have
TASK-10	Full-text search across task titles and descriptions within a workspace.	Should have

4.5 Real-Time Collaboration

ID	Requirement	Priority
RT-01	When a user opens a project board, they join a Socket.io room for that project.	Must have
RT-02	When any member moves a task, all other members in the same project room see the card move within 500ms.	Must have
RT-03	When a task's title, assignee, or priority changes, the update is pushed to all room members in real time.	Must have
RT-04	When a new comment is posted, it appears for all users viewing that task without refresh.	Must have
RT-05	When a new task is created, it appears on all open boards in that project.	Must have
RT-06	Users leave the project room when they navigate away. Server-side cleanup on disconnect.	Must have
RT-07	Show live presence indicators: avatars of users currently viewing the same board.	Nice to have

4.6 Notifications

ID	Requirement	Priority
NOTIF-01	User receives an in-app notification when they are assigned to a task.	Must have
NOTIF-02	User receives an in-app notification when they are @mentioned in a comment.	Must have
NOTIF-03	Notification panel shows unread count badge on the bell icon.	Must have
NOTIF-04	User can mark individual notifications as read, or mark all as read.	Must have
NOTIF-05	Notifications are delivered in real time via Socket.io to the recipient if they are online.	Must have

4.7 Role-Based Access Control (RBAC)

Action	Owner	Member	Viewer
View workspace, projects, tasks	Yes	Yes	Yes
Create / edit tasks	Yes	Yes	No
Delete tasks	Yes	Own only	No
Create / delete projects	Yes	Yes*	No

Action	Owner	Member	Viewer
Manage project columns	Yes	No	No
Invite / remove members	Yes	No	No
Change member roles	Yes	No	No
Delete workspace	Yes	No	No

Note: * Member can delete only projects they personally created.

5. Data Model

5.1 Entity Relationship Overview

The following entities form the core data model. All tables include created_at and updated_at timestamps (managed by Prisma).

5.2 Schema Definitions

Users

Column	Type	Constraints	Notes
id	UUID	PK, default gen_random_uuid()	
name	VARCHAR(100)	NOT NULL	
email	VARCHAR(255)	NOT NULL, UNIQUE	
password_hash	VARCHAR(255)	NOT NULL	bcrypt hash
avatar_initials	VARCHAR(4)	nullable	Auto-generated from name
created_at	TIMESTAMP TZ	NOT NULL, default now()	
updated_at	TIMESTAMP TZ	NOT NULL	Managed by Prisma

Workspaces

Column	Type	Constraints	Notes
id	UUID	PK	
name	VARCHAR(100)	NOT NULL	
slug	VARCHAR(100)	NOT NULL, UNIQUE	URL-safe identifier, auto-generated
owner_id	UUID	FK -> users.id	The original creator
created_at	TIMESTAMP TZ	NOT NULL	

Workspace Members (join table)

Column	Type	Constraints	Notes
workspace_id	UUID	FK -> workspaces.id	Composite PK with user_id
user_id	UUID	FK -> users.id	Composite PK with workspace_id
role	ENUM	NOT NULL	owner member viewer
joined_at	TIMESTAMPTZ	NOT NULL	

Projects

Column	Type	Constraints	Notes
id	UUID	PK	
workspace_id	UUID	FK -> workspaces.id, NOT NULL	Cascade delete
name	VARCHAR(100)	NOT NULL	
description	TEXT	nullable	
colour	VARCHAR(7)	nullable	Hex colour code e.g. #FF5733
created_by	UUID	FK -> users.id	
created_at	TIMESTAMPTZ	NOT NULL	

Columns (status columns per project)

Column	Type	Constraints	Notes
id	UUID	PK	
project_id	UUID	FK -> projects.id, NOT NULL	Cascade delete
name	VARCHAR(100)	NOT NULL	e.g. 'In Progress'
order_index	INTEGER	NOT NULL	For column ordering

Tasks

Column	Type	Constraints	Notes
id	UUID	PK	
project_id	UUID	FK -> projects.id, NOT NULL	

Column	Type	Constraints	Notes
column_id	UUID	FK -> columns.id, NOT NULL	Current status column
title	VARCHAR(500)	NOT NULL	
description	TEXT	nullable	Markdown content
priority	ENUM	NOT NULL, default 'none'	none low medium high urgent
assignee_id	UUID	FK -> users.id, nullable	
due_date	DATE	nullable	
order_key	VARCHAR(255)	NOT NULL	LexoRank string for ordering within column
created_by	UUID	FK -> users.id	
created_at	TIMESTAMPZ	NOT NULL	
updated_at	TIMESTAMPZ	NOT NULL	

Labels

Column	Type	Constraints	Notes
id	UUID	PK	
workspace_id	UUID	FK -> workspaces.id	Labels are workspace-scoped
name	VARCHAR(50)	NOT NULL	
colour	VARCHAR(7)	NOT NULL	Hex colour

Task Labels (join table)

Column	Type	Constraints	Notes
task_id	UUID	FK -> tasks.id, Composite PK	Cascade delete
label_id	UUID	FK -> labels.id, Composite PK	

Comments

Column	Type	Constraints	Notes
id	UUID	PK	
task_id	UUID	FK -> tasks.id, NOT NULL	Cascade delete
author_id	UUID	FK -> users.id, NOT NULL	
body	TEXT	NOT NULL	Markdown. Store raw markdown.
created_at	TIMESTAMP Z	NOT NULL	
updated_at	TIMESTAMP Z	NOT NULL	Track edits

Activity Log

Column	Type	Constraints	Notes
id	UUID	PK	
task_id	UUID	FK -> tasks.id, NOT NULL	
actor_id	UUID	FK -> users.id, NOT NULL	Who performed the action

Column	Type	Constraints	Notes
action	VARCHAR(100)	NOT NULL	e.g. 'changed_status', 'assigned', 'commented'
meta	JSONB	nullable	Stores before/after values e.g. {from: 'Todo', to: 'Done'}
created_at	TIMESTAMPTZ	NOT NULL	

Notifications

Column	Type	Constraints	Notes
id	UUID	PK	
recipient_id	UUID	FK -> users.id, NOT NULL	
type	ENUM	NOT NULL	assigned mentioned
task_id	UUID	FK -> tasks.id, NOT NULL	The related task
actor_id	UUID	FK -> users.id	Who triggered the notification
is_read	BOOLEAN	NOT NULL, default false	
created_at	TIMESTAMPTZ	NOT NULL	

6. API Contract

All endpoints are prefixed with /api/v1. All request and response bodies are JSON. All authenticated endpoints require Authorization: Bearer <access_token> header.

Standard error response shape: { error: string, code: string }. Standard success shape for lists: { data: T[], meta: { total, page, limit } }.

6.1 Authentication Endpoints

Meth od	Path	Auth Required	Description
POST	/auth/register	No	Register new user. Body: { name, email, password }.
POST	/auth/login	No	Login. Returns access token in body, refresh token as HTTP-only cookie.
POST	/auth/refresh	No (cookie)	Uses refresh token cookie to issue new access token.
POST	/auth/logout	Yes	Invalidates refresh token in Redis.
GET	/auth/me	Yes	Returns current authenticated user profile.

6.2 Workspace Endpoints

Meth od	Path	Role Required	Description
POST	/workspaces	Authenticated	Create a new workspace.
GET	/workspaces	Authenticated	List all workspaces the current user belongs to.
GET	/workspaces/:wsId	Member+	Get workspace details.
PATCH	/workspaces/:wsId	Owner	Update workspace name.
DELETE	/workspaces/:wsId	Owner	Delete workspace and all contents.
GET	/workspaces/:wsId/members	Member+	List all members with their roles.
POST	/workspaces/:wsId/members	Owner	Invite a user by email. Body: { email, role }.

Method	Path	Role Required	Description
PATCH	/workspaces/:wsId/members/:userId	Owner	Change a member's role.
DELETE	/workspaces/:wsId/members/:userId	Owner	Remove a member from workspace.

6.3 Project Endpoints

Method	Path	Role Required	Description
POST	/workspaces/:wsId/projects	Member+	Create a project.
GET	/workspaces/:wsId/projects	Member+	List all projects in the workspace.
GET	/workspaces/:wsId/projects/:projectId	Member+	Get project detail including columns.
PATCH	/workspaces/:wsId/projects/:projectId	Member+	Update project name, description, colour.
DELETE	/workspaces/:wsId/projects/:projectId	Owner/Creator	Delete project.
GET	/workspaces/:wsId/projects/:projectId/columns	Member+	List columns for a project.
POST	/workspaces/:wsId/projects/:projectId/columns	Owner	Add a column.
PATCH	/workspaces/:wsId/projects/:projectId/columns/:colId	Owner	Rename or reorder a column.
DELETE	/workspaces/:wsId/projects/:projectId/columns/:colId	Owner	Delete a column (tasks must be moved first).

6.4 Task Endpoints

Method	Path	Role Required	Description
GET	/projects/:projectId/tasks	Member+	List all tasks. Supports query params: assignee, priority, label, due_before, due_after, search.
POST	/projects/:projectId/tasks	Member+	Create a task.
GET	/projects/:projectId/tasks/:taskId	Member+	Get full task detail including comments and activity log.

Method	Path	Role Required	Description
PATCH	/projects/:projectId/tasks/:taskId	Member+	Update any task field. Triggers activity log entry and Socket.io broadcast.
PATCH	/projects/:projectId/tasks/:taskId/move	Member+	Move task to a different column or reorder. Body: { columnId, orderKey }.
DELETE	/projects/:projectId/tasks/:taskId	Owner/Creator	Delete task.
POST	/projects/:projectId/tasks/:taskId/comments	Member+	Add a comment. Body: { body }.
PATCH	/projects/:projectId/tasks/:taskId/comments/:commentId	Comment author	Edit own comment.
DELETE	/projects/:projectId/tasks/:taskId/comments/:commentId	Owner/Author	Delete comment.
GET	/projects/:projectId/tasks/:taskId/activity	Member+	Get activity log for a task.

6.5 Notification Endpoints

Method	Path	Description
GET	/notifications	Get current user's notifications. Supports query: ?unread=true.
PATCH	/notifications/:notificationId/read	Mark a notification as read.
PATCH	/notifications/read-all	Mark all notifications as read.

7. Real-Time Events

Socket.io is used for real-time communication. The client connects on authentication, then joins rooms based on navigation.

7.1 Room Strategy

- workspace:{workspaceId} — joined on workspace load. Used for notification delivery.
- project:{projectId} — joined when user opens a project board. Used for task updates, column changes, and presence.
- Rooms are joined with socket.join() on the server side when the client emits a join event.

7.2 Client-to-Server Events

Event Name	Payload	Description
join:project	{ projectId: string }	Join the project room when user navigates to a board.
leave:project	{ projectId: string }	Leave the project room on navigation away.
join:workspace	{ workspaceId: string }	Join workspace room for notifications.

7.3 Server-to-Client Events

Event Name	Payload	Trigger
task:created	Full task object	New task created in the project
task:updated	{ taskId, changes: Partial<Task> }	Any task field updated
task:moved	{ taskId, columnId, orderKey }	Task dragged to new column or position
task:deleted	{ taskId }	Task deleted
comment:created	Full comment object with author	Comment posted on any task in project
column:created	Full column object	New column added to project
column:updated	{ columnId, changes }	Column renamed or reordered
column:deleted	{ columnId }	Column deleted
notification:new	Full notification object	Sent to individual user's workspace room
presence:update	{ userId, name, avatarInitials, action: 'join' 'leave' }	User joins or leaves project board

8. Non-Functional Requirements

Category	Requirement
Performance	API endpoints (excluding search) must respond within 300ms at p95 under typical single-user load.
Performance	Real-time events must be broadcast to room members within 500ms of the originating action.
Security	Passwords stored as bcrypt hashes. No plaintext credentials anywhere in code or logs.
Security	JWT access tokens expire in 15 minutes. Refresh tokens expire in 7 days and are invalidated on logout.
Security	All API inputs validated and sanitised. Use a validation library (Zod recommended) on both client and server.
Security	CORS configured to allow only the frontend origin. Not wildcard (*) in production.
Security	SQL injection prevented by Prisma's parameterised queries. No raw SQL with user input.
Reliability	Docker Compose setup must allow any developer to run the full stack with a single command.
Reliability	Database schema managed with Prisma migrations. No manual schema changes.
Code Quality	TypeScript strict mode enabled on both frontend and backend. No use of any where avoidable.
Code Quality	ESLint + Prettier configured. CI fails if lint errors or type errors are present.
Code Quality	Environment variables documented in .env.example files. No secrets committed to git.
Usability	Application must be fully functional on latest Chrome, Firefox, and Safari.
Usability	Loading states, error states, and empty states handled in the UI for all async operations.

9. Implementation Plan

This plan is designed so that you always have a working, deployable application at the end of each phase. Do not move to the next phase until the current one is solid.

Phase 1 — Foundation (Week 1–2)

- Set up monorepo folder structure
- Write docker-compose.yml with postgres, redis, backend, frontend services
- Initialise Express + TypeScript backend. Set up Prisma with initial User and Workspace schema.
- Implement AUTH-01 through AUTH-06: register, login, refresh, logout, auth middleware
- Initialise React + Vite + TypeScript frontend. Configure Tailwind and React Query.
- Build login and register pages. Connect to auth API. Store access token in memory (not localStorage).
- Build protected route wrapper. Verify auth flow end-to-end.

***Note:** Read: Prisma Getting Started docs, Express TypeScript setup guide, React Query Quick Start. Do not skip the Prisma docs — understanding migrations and the Prisma Client is essential.*

Phase 2 — Core Data (Week 2–3)

- Add Workspace, Project, Column, Task, and WorkspaceMember models to Prisma schema. Run migrations.
- Build workspace CRUD endpoints. Implement RBAC middleware.
- Build project CRUD endpoints.
- Build task CRUD endpoints (no real-time yet). Implement the activity log on task mutations.
- Build workspace and project list pages in the frontend. Implement Zustand store for active workspace/project.
- Build task creation form. Display tasks as a simple list (kanban layout comes next phase).

Phase 3 — Kanban Board (Week 3–4)

- Install and learn dnd-kit. Read their sortable and multiple containers examples in the docs.
- Implement the full kanban board layout: columns, task cards, drag between columns.
- Connect drag events to PATCH /tasks/:id/move API. Implement optimistic updates with React Query.
- Implement LexoRank or a simple fractional indexing for task order persistence.
- Build task detail modal/sidebar: all fields, edit in place, comments section, activity log.

Phase 4 — Real-Time (Week 4–5)

- Install Socket.io on the backend. Attach to the same Express HTTP server.

- Set up socket authentication middleware: validate the access token on socket handshake.
- Implement join:project and leave:project room events.
- Add Socket.io broadcasts to task mutation endpoints (created, updated, moved, deleted).
- On the frontend, initialise the socket client after login. Subscribe to project room events and update React Query cache directly on socket events (do not refetch — update the cache manually).
- Test real-time with two browser windows. Verify cards move without refresh.

Note: This is the most technically challenging phase. Read the Socket.io server docs thoroughly, especially the rooms and authentication sections. The key insight is that on the frontend, socket events should call `queryClient.setQueryData()` to update the cache — not trigger a new HTTP request.

Phase 5 — Notifications and Search (Week 5–6)

- Implement notification creation on task assignment and @mention parsing in comments.
- Deliver notifications via Socket.io to the recipient's workspace room.
- Build notification panel in the UI with unread count badge.
- Implement task search endpoint using PostgreSQL full-text search (`to_tsvector` / `to_tsquery`).
- Build filter UI on the kanban board: filter by assignee, priority, label, due date.

Phase 6 — Polish and DevOps (Week 6–8)

- Add GitHub Actions CI workflow: lint, typecheck, and run Prisma migrate deploy on push to main.
- Write `docker-compose.prod.yml`. Test production build locally with Docker.
- Deploy to a free tier host (Render, Railway, or Fly.io work well for this stack).
- Handle all loading states, error boundaries, empty states, and 404 pages in the frontend.
- Write a thorough README.md: what the app does, how to run it locally, architecture overview, environment variables list.
- Record a short demo video (2–3 minutes) showing the real-time features. This is important for your portfolio — interviewers may not run it locally.

10. Key Learning Resources

For each technology you are learning for the first time, read the official documentation before anything else. These are the specific sections most relevant to this project:

Technology	What to Read First	URL (search these)
Zustand	Getting Started + Guides: Slices Pattern, Practice with No Store Actions	zustand docs
React Query	Quick Start, Queries, Mutations, Optimistic Updates, and the QueryClient section	tanstack query docs
Socket.io	Server: Rooms, Middlewares, Handling CORS. Client: Initialization, Events.	socket.io docs
Prisma	Getting Started, Data Modeling, Migrations, CRUD, and Relations	prisma docs
dnd-kit	Sortable preset, Multiple Containers example	dnd-kit docs
Docker Compose	Compose file reference, Networking in Compose, Use volumes	docker compose docs
Redis (ioredis)	Basic commands, Key expiry, Using ioredis with Node.js	ioredis github

Note: Do not watch YouTube tutorials as your primary learning source for this project. Documentation is denser and faster. Use AI assistance to ask questions about specific concepts you have read but not fully understood — not to write code you haven't engaged with yet.

End of Document — TaskFlow SRS v1.0